

TITLE: SOFTWARE
DEVELOPERS SALARY
PREDICTION.

NAME: SABITHA G

ROLL NO: E25CG036

PROJECT TITLE: SOFTWARE
DEVELOPERS SALARY
PREDICTIONS

COURSE : BSC COMPUTER
SCIENCE WITH COGNITIVE
SYSTEM



TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Dataset Description

4.1 Data Collection – Input

4.2 Data Collection – Output

5. Data Understanding

5.1 Data Information – df.shape

5.2 Data Information – df.info()

5.3 Data Information – df.describe()

6. Data Preprocessing

6.1 Handling Duplicate Values

6.2 Handling Inconsistent Values

6.3 Handling Missing Values

7. Exploratory Data Analysis (EDA)

7.1 Correlation Analysis

7.2 Experience vs Salary Analysis

7.3 Salary Distribution

7.4 Outlier Detection

8. Feature Engineering

9. Model Building

9.1 Train-Test Split

9.2 Feature Scaling – Input

9.3 Feature Scaling – Output

9.4 Linear Regression Model

10. Model Evaluation

10.1 Actual vs Predicted – Input

10.2 Actual vs Predicted – Output

10.3 Evaluation Metrics – Input

10.4 Evaluation Metrics – Output

10.5 Final Prediction

11. Results and Interpretation

12. Conclusion

13. Future Scope

14. References

1. Abstract

This project focuses on developing a data-driven predictive analytics system to analyze developer salary datasets and predict salary outcomes. The system includes data preprocessing, exploratory data analysis (EDA), and machine learning model development. Various factors such as experience, job role, and location were analyzed to understand their impact on salary. Regression models were applied to predict salaries with reasonable accuracy. The project provides meaningful insights and supports data-driven decision-making.

2. Introduction

Predictive analytics uses data, statistical algorithms, and machine learning techniques to identify patterns and predict future outcomes. In this project, developer salary prediction is performed using structured datasets.

Understanding salary trends helps individuals and organizations make better decisions regarding hiring, career planning, and compensation strategies.

3. Problem Statement

Design and develop a data-driven predictive analytics system that analyzes datasets to identify key influencing factors and accurately predict outcomes using preprocessing, EDA, and machine learning techniques.

4. Objectives

- To analyze the dataset using preprocessing and EDA
- To identify important factors affecting salary
- To build machine learning models for prediction
- To evaluate model performance

5.Dataset Description

The dataset used in this project contains information about developer salaries along with various influencing features.

Features include:

- Experience level
- Job role
- Location
- Salary

6.Importing Libraries

The project begins with importing the necessary Python libraries required for data analysis, visualization, and machine learning. Libraries such as Pandas and NumPy are used for data manipulation, while Matplotlib and Seaborn are used for data visualization. Scikit-learn is utilized for building and evaluating the machine learning model.

```
[1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

7.Data Collection

7.1 Data Collection – Input

The dataset is loaded into the environment using appropriate functions. The data is typically stored in a CSV file format and is read using Pandas.

```
[2]: df = pd.read_csv("train.csv")

df.head()
```

7.2 Data Collection – Output

After loading, the dataset is displayed to verify successful import and to understand its structure.

```
[2]:
```

	experience	country	education	languages	frameworks	company_size	salary_usd
0	34	Germany	Masters	Rust, JavaScript	Vue, ASP.NET	201-1000	172292
1	9	UK	Masters	C#, JavaScript	Ruby on Rails, Spring	201-1000	82311
2	8	USA	Some College	C++, Go	Vue, React	5000+	109637
3	29	USA	Masters	C++, Ruby	Spring, Angular	11-50	214386
4	7	USA	Bachelors	C#, Rust	React, Express	1-10	107584

8. Data Understanding

8.1 Data Information – df.describe()

The df.describe() function generates statistical summaries such as mean, median, standard deviation, and quartiles for numerical columns.

```
[5]: df.describe()
```

```
[5]:
```

	experience	salary_usd
count	40000.000000	40000.000000
mean	19.912875	131834.441525
std	11.865220	46426.249810
min	0.000000	12024.000000
25%	10.000000	98172.250000
50%	20.000000	130579.000000
75%	30.000000	164431.000000
max	40.000000	277554.000000

8.2.Data Information – df.shape

The shape of the dataset is analyzed to determine the number of rows and columns present.

```
[3]: df.shape
```

```
[3]: (40000, 7)
```

8.3.Data Information – df.info()

The df.info() function provides details about data types, non-null values, and memory usage.

```
[4]: df.info()
```

```
<class 'pandas.DataFrame'>  
RangeIndex: 40000 entries, 0 to 39999  
Data columns (total 7 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   experience             40000 non-null  int64  
1   country                40000 non-null  str  
2   education              40000 non-null  str  
3   languages              40000 non-null  str  
4   frameworks             40000 non-null  str  
5   company_size           40000 non-null  str  
6   salary_usd             40000 non-null  int64  
dtypes: int64(2), str(5)  
memory usage: 2.1 MB
```


9.Data Preprocessing

9.1 Handling Duplicate Values

Duplicate records in the dataset are identified and removed to ensure data integrity and avoid bias in model training.

```
[7]: df.duplicated().sum()
```

```
[7]: np.int64(0)
```

9.2. Handling Inconsistent Values

Inconsistent or incorrect data entries are corrected to maintain uniformity across the dataset.

```
[8]: df['country'].unique()  
df['education'].unique()  
df['company_size'].unique()
```

```
[8]: <StringArray>  
['201-1000', '5000+', '11-50', '1-10', '51-200', '1001-5000']  
Length: 6, dtype: str
```

9.3. Handling Missing Values

Missing values are handled using appropriate techniques such as removal or imputation, ensuring the dataset is complete for analysis.

```
[6]: df.isnull().sum()
```

```
[6]: experience      0  
country           0  
education         0  
languages         0  
frameworks       0  
company_size     0  
salary_usd       0  
dtype: int64
```

10. Exploratory Data Analysis (EDA)

EDA is performed to uncover patterns, relationships, and trends within the dataset.

10.1 Correlation Analysis

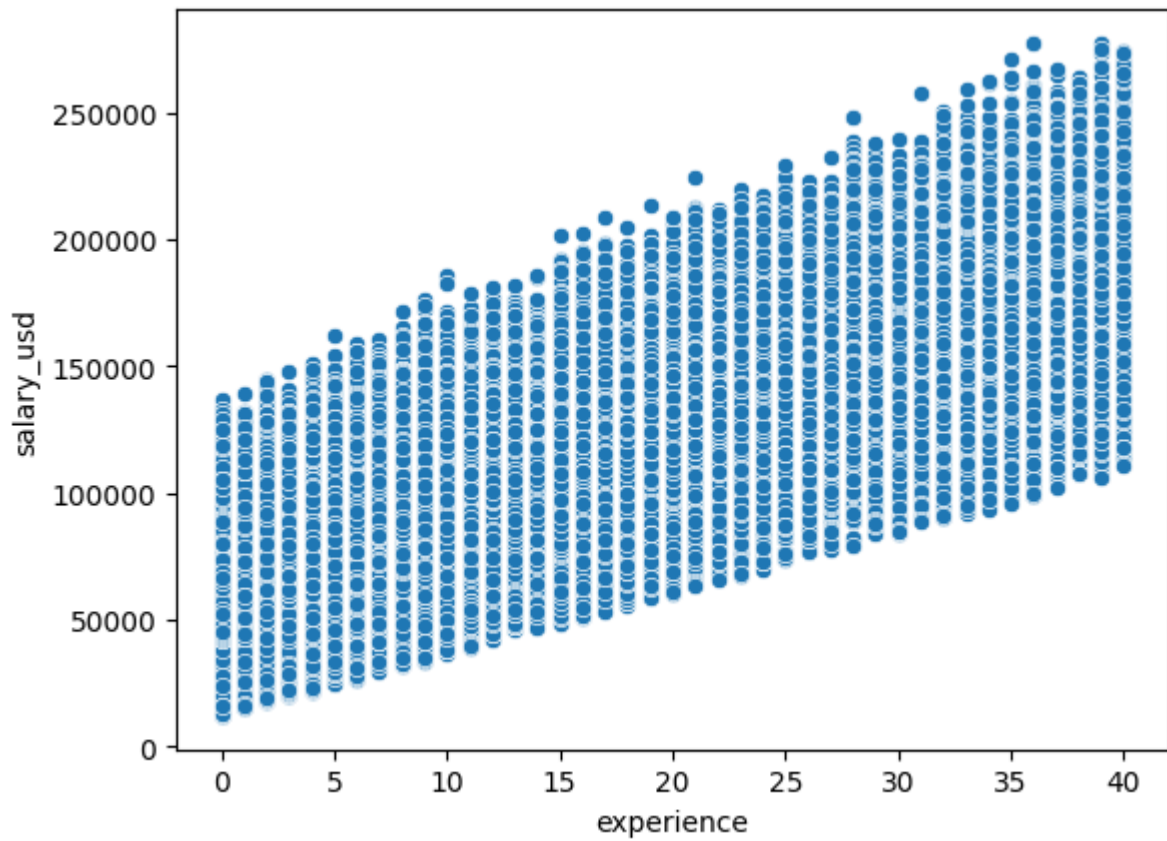
Correlation analysis is used to identify relationships between variables. A heatmap is used to visualize the correlation matrix.

```
[12]: num_df = df.select_dtypes(include=['int64', 'float64'])  
  
plt.figure(figsize=(6,4))  
sns.heatmap(num_df.corr(), annot=True)  
plt.show()
```



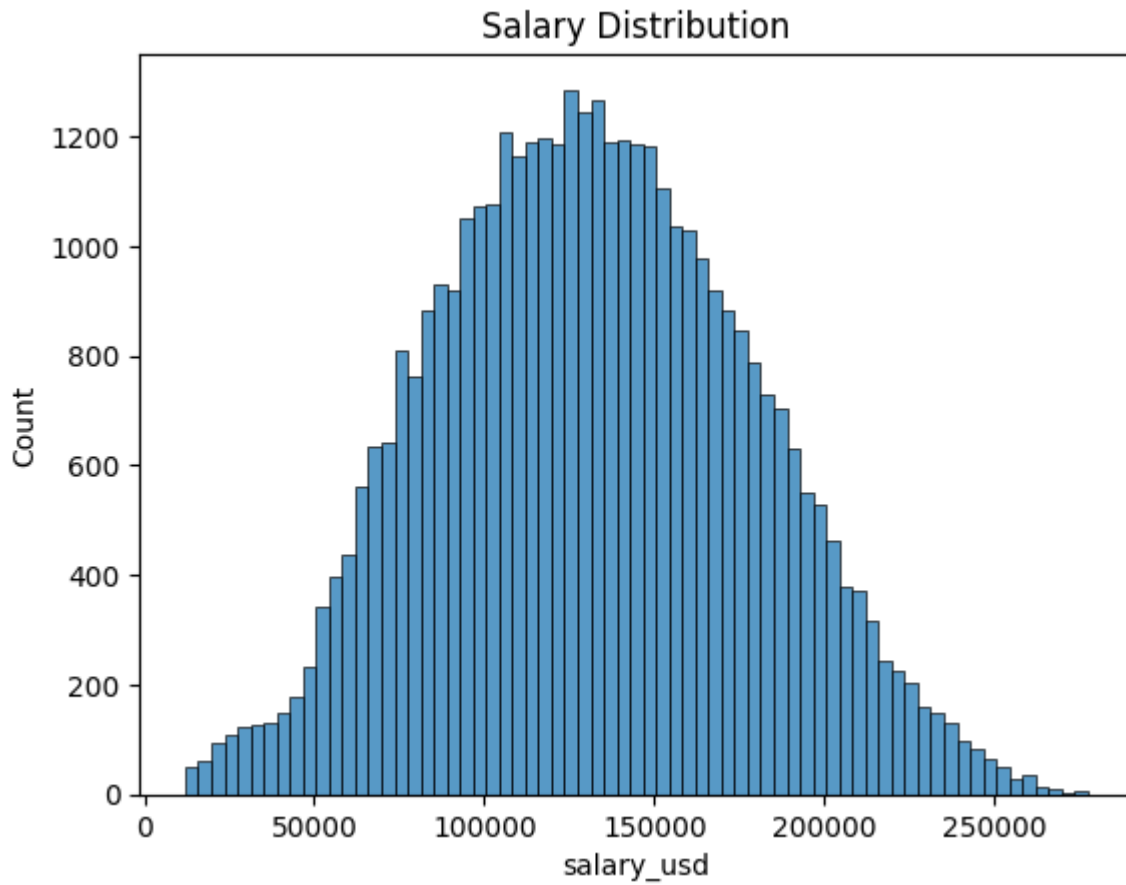
10.2 Experience vs Salary Analysis

A graphical analysis is performed to understand how experience impacts salary. It shows a positive relationship between experience and salary.



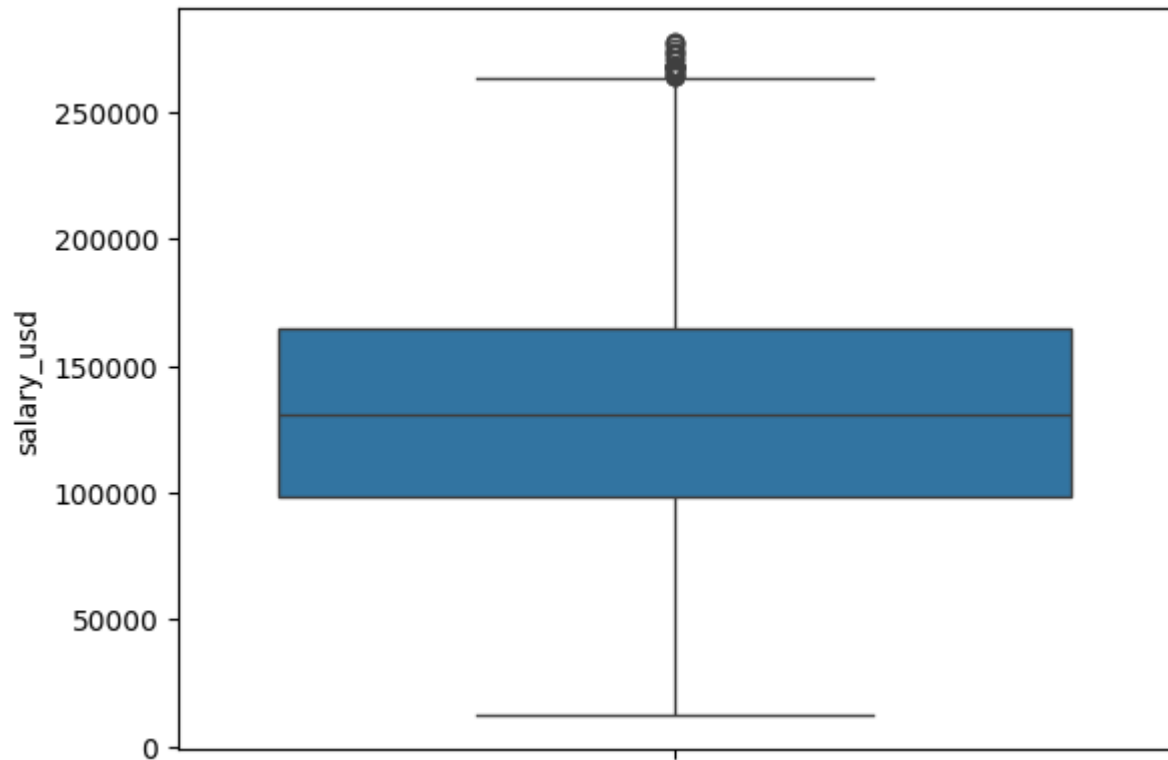
10.3 Salary Distribution

The distribution of salary is analyzed to understand its spread and frequency across different ranges.



10.4 Outlier Detection

Outliers in the dataset are identified using visualization techniques such as boxplots to ensure accurate model performance.



11. Feature Engineering

Relevant features are selected and transformed to improve model performance. Irrelevant or redundant features are removed.

```
[13]: df['salary_per_experience'] = df['salary_usd'] / (df['experience'] + 1)
df.head()
```

```
[13]:
```

	experience	country	education	languages	frameworks	company_size	salary_usd	salary_per_experience
0	34	Germany	Masters	Rust, JavaScript	Vue, ASP.NET	201-1000	172292	4922.628571
1	9	UK	Masters	C#, JavaScript	Ruby on Rails, Spring	201-1000	82311	8231.100000
2	8	USA	Some College	C++, Go	Vue, React	5000+	109637	12181.888889
3	29	USA	Masters	C++, Ruby	Spring, Angular	11-50	214386	7146.200000
4	7	USA	Bachelors	C#, Rust	React, Express	1-10	107584	13448.000000

12. Model Building

12.1 Train-Test Split

The dataset is divided into training and testing sets to evaluate the model's performance on unseen data

```
[14]: from sklearn.model_selection import train_test_split
```

```
X = df.drop('salary_usd', axis=1)
y = df['salary_usd']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (32000, 7)
X_test shape: (8000, 7)
y_train shape: (32000,)
y_test shape: (8000,)
```

12.2 Feature Scaling – Input

Feature scaling is applied to normalize the input data, ensuring all features contribute equally to the model.

```
[15]: #drop non-numeric before scaling  
X_train = X_train.select_dtypes(include=np.number)  
X_test = X_test.select_dtypes(include=np.number)
```

```
[17]: from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

```
[18]: pd.DataFrame(X_train).head()
```

12.3 Feature Scaling – Output

The scaled data is obtained and used for model training.

```
[18]:
```

	0	1
0	-0.665577	-0.391364
1	0.262130	-0.530470
2	1.611523	-0.417554
3	-0.496903	-0.410510
4	0.768152	-0.263867

11.4 Linear Regression Model

A Linear Regression model is implemented to predict salary based on input features. The model is trained using the training dataset.

```
[19]: from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()  
model.fit(X_train, y_train)
```

```
[19]: ▾ LinearRegression ⓘ ?  
      ▶ Parameters
```

13. Model Evaluation

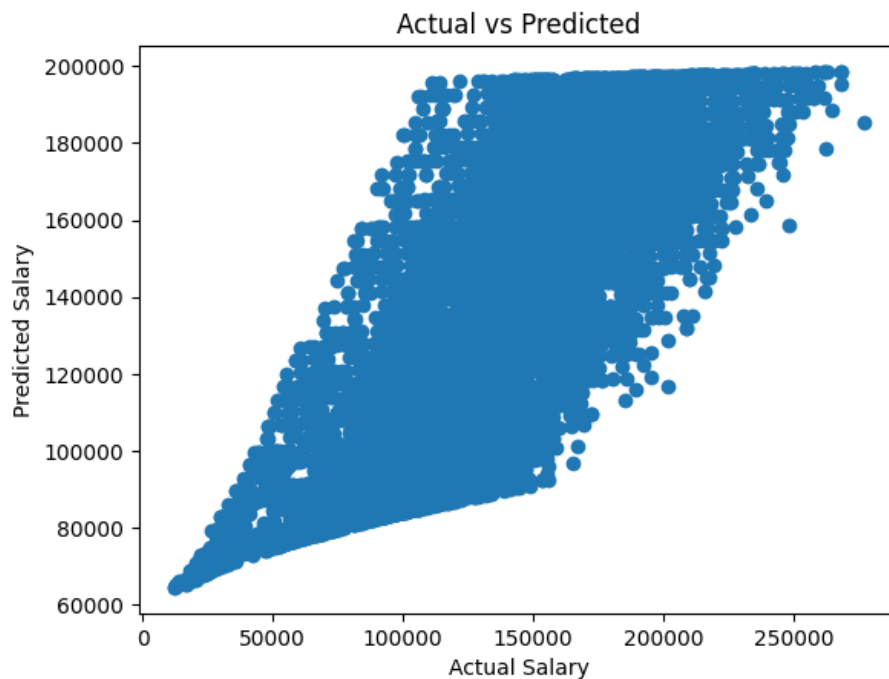
13.1 Actual vs Predicted – Input

The actual and predicted values are compared to evaluate model accuracy

```
[22]: plt.scatter(y_test, y_pred)  
      plt.xlabel("Actual Salary")  
      plt.ylabel("Predicted Salary")  
      plt.title("Actual vs Predicted")  
      plt.show()
```


13.2 Actual vs Predicted – Output

A graphical representation is used to compare actual and predicted values.



13.3 Evaluation Metrics – Input

Evaluation metrics are defined to measure model performance.

```
[21]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

12.4 Evaluation Metrics – Output

Metrics such as R^2 Score, Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) are calculated.

```
MAE: 22965.03913317421
MSE: 815323247.9883074
RMSE: 28553.86572757369
R2 Score: 0.6212758316357456
```

13.5 Final Prediction

The trained model is used to make predictions on new or test data.

```
[20]: y_pred = model.predict(X_test)
      y_pred[:5]
```

```
[20]: array([ 88247.81085164, 169022.86598273, 125620.35987563,  73137.8011991 ,
            82142.83246796])
```

13. Results and Interpretation

The analysis shows that experience and role significantly influence salary. The model demonstrates good predictive capability and provides reliable results for salary estimation.

14. Conclusion

This project successfully developed a predictive analytics system for developer salary prediction. Data preprocessing, EDA, and machine learning techniques were effectively applied. The model provides meaningful insights and supports decision-making.

15. Future Scope

- Implementation of advanced machine learning models
- Inclusion of more diverse datasets
- Deployment as a web-based application

16. References

- Kaggle
- Python Libraries:
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - Scikit-learn